

Learning Mocha

A local-first personal knowledge base
with an AI learning assistant

CS426 — Mobile Device Application Development
Final Project Report

Student ID	Full name
24125006	Nguyen Anh Dung
24125009	Le Huu Hoa

Platform	Native Android (Kotlin + Java, XML Views), API 24+
Repository	app/ Android application, backend/ DeepSeek gateway
Deliverables	signed release APK, source with git history, this report, demo video
Submitted	5 September 2026

Everything the user writes stays on the device. The only data that leaves it is the text the user chooses to send to the AI assistant.

Contents

1	The project	3
1.1	The problem	3
1.2	Learning Mocha	3
1.3	The two commitments that shaped the design	3
1.4	Why this is not a template project	3
2	Target users	4
2.1	Who this is for	4
2.2	Three concrete users	4
2.3	Who this is not for	4
2.4	What the user has to do, and what the app does for them	5
3	Features	5
3.1	The knowledge base	5
3.2	Reading and writing	5
3.3	Connecting knowledge	6
3.4	Learning metadata	6
3.5	Dictionary	7
3.6	Finding things	7
3.7	The AI assistant	7
3.8	Backup, settings and theming	7
3.9	Offline and degraded states	8
4	Application architecture	8
4.1	The system in one picture	8
4.2	Layers on the device	9
4.3	Why some of it is Java	10
4.4	Threading, lifecycle, and what survives what	10
4.5	Navigation	10
4.6	The invariant	11
5	Data model and storage	11
5.1	Why a database and not files	11
5.2	One table for the tree	12
5.3	The satellite tables	12
5.4	Full-text search: <code>posts_fts</code>	13
5.5	Migrations and schema history	13
5.6	Settings	14
5.7	The backup file	14
6	The AI learning assistant	14
6.1	What makes it more than a chatbot	14
6.2	Four modes	15
6.3	The gateway	15
6.4	Choosing what the model may see	16
6.5	The action protocol	17
6.6	Validation, review and undo	17
6.7	Streaming	18
6.8	Privacy	19
6.9	Where the code differs from the plan	19
7	Technologies used	19
7.1	The stack, and why each piece is there	19
7.2	Deliberate non-dependencies	21
7.3	Development tooling	21

8	Setup and installation	21
8.1	Installing the shipped APK	21
8.2	Building the Android app from source	21
8.3	Running the AI gateway	22
8.4	Rebuilding this report	22
8.5	Assembling the submission archive	22
9	Work division	23
9.1	Shared work	23
9.2	Commit history	23
10	Testing and verification	24
10.1	The decision that makes the unit suite worth having	24
10.2	JVM unit tests	24
10.3	Instrumented tests: Room migrations	25
10.4	Running the suites	25
10.5	Manual verification on the emulator	25
10.6	What is not covered	26
11	Self-assessment	26
11.1	What was completed	26
11.2	What was deliberately not built	27
11.3	Known limits of features that do ship	27
11.4	Where the code diverged from the plan	27
11.5	What a final review pass found	28
11.6	If there were more time	28
A	Screen gallery	28

1 The project

1.1 The problem

People who learn technical subjects on their own accumulate notes faster than they accumulate understanding. The notes end up scattered across a chat with themselves, a folder of screenshots, twelve open browser tabs and three note-taking apps, and the connective tissue — *this depends on that, I already know this part, that term means something specific here* — lives only in memory and decays. Six months later the notes are still there and the thread through them is gone.

Two things are missing from the usual note app. The first is **structure that survives**: a way to say that *Raft* belongs under *Consensus* under *Distributed Systems*, that it relates to *Leader Election*, and that it is currently half-read rather than finished. The second is **help with the part that is actually hard**: not writing the notes down, but working out what to learn next, in what order, given what is already understood.

1.2 Learning Mocha

Learning Mocha is a native Android app that treats a person's own learning as a small private encyclopaedia they own outright:

- a **personal Wikipedia** — Markdown articles organised into branches, folders and subfolders, cross-linked with `[[wiki-links]]` that resolve to real posts and produce backlinks automatically;
- a **learning tracker** — per-post status, favourites, tags, external references and YouTube resources that play without leaving the article, and a glossary that defines a term in English and Vietnamese without interrupting the reading;
- a **knowledge graph** — the same library seen as nodes and edges, so gaps and clusters are visible at a glance rather than inferred from a file tree;
- an **AI learning assistant** — a chat that can read the library through explicit, read-only queries and *propose* concrete changes: generate an article, build a learning path as a tree of posts, link things together, reorganise a branch. Every proposal is reviewed item by item before anything is written.

1.3 The two commitments that shaped the design

Almost every interesting decision in this project follows from two commitments made before any code was written.

1. Local-first, not offline-tolerant

The knowledge base lives in a Room database on the device and nowhere else. There is no account, no sync, no cloud copy, and Android's own Auto Backup is switched off so the operating system does not quietly upload the database either. Consequently every feature except the assistant works with the network switched off — not degraded, but identical — and the app cannot lose someone's library because a server went away. The cost is real and accepted: no multi-device access, and backup is the user's responsibility, which is why export/import is a first-class screen with a weekly reminder.

2. The AI never writes to the database

A language model that can call `DELETE` is a data-loss feature. Instead the model has no database access at all: it can ask for information through seven read-only queries, and it can *propose* changes only as a batch of structured JSON actions. The device parses that batch, validates it against live data, shows it on a review screen where each action can be inspected and deselected, and then applies the approved subset in a single Room transaction. Destructive actions arrive unchecked and need an explicit confirmation. This is what makes an assistant with write ambitions safe enough to actually use.

1.4 Why this is not a template project

The parts that took the most thought are the parts that do not exist in a tutorial: a single self-referencing table that makes branches, folders and posts share one set of tree operations; a content-backed FTS4 index kept in step by Room's own triggers; wiki-link resolution that survives renaming a post; a model-independent action protocol with on-device validation and single-level undo; a two-round context mechanism that lets

the model query the library without ever being sent it; and a force-directed graph renderer written against **Canvas** with the layout maths in framework-free Java so it can be unit-tested. §11 is honest about what is and is not finished.

2 Target users

2.1 Who this is for

Learning Mocha is built for the **self-directed technical learner**: a student or working developer who reads documentation, courses, papers and videos on their own initiative and wants the result to accumulate into something they can navigate a year later. Concretely, the app assumes a user who

- studies subjects with real internal structure — programming, systems, mathematics, languages — where topic B genuinely depends on topic A;
- writes their own notes rather than collecting other people's, and is comfortable with Markdown;
- cares that their notes stay theirs: no account, no vendor, no silent upload;
- reads on a phone in short sessions, so “where was I” and “what should I read next” matter more than word-processing features.

2.2 Three concrete users

Minh — second-year CS student

Studying for a distributed-systems course. He types notes during lectures and wants them to end up in the right place in a tree he can revise from. He uses **status** to mark which lecture notes are still half-understood, the **glossary** because half the vocabulary is new, and he asks the assistant “based on what I already have, what am I missing before Raft?” The answer arrives as a proposed tree of posts he can prune before accepting.

Lan — backend developer changing stack

Moving from Node to Spring Boot. She keeps a *Backend* branch and writes a post per concept as she hits it, linking `[[Dependency Injection]]` from five different articles. **Backlinks** tell her which of her own notes depend on a concept she is about to revise; the **knowledge graph** shows her that her *Java* cluster is dense and her *Deployment* cluster is three isolated dots. She uses **Organize** mode to ask the assistant to tidy a branch that grew by accretion.

Hoa — learning a language and reading papers

Reads English-language material and stores Vietnamese meanings alongside definitions, which is why the glossary has a dedicated **meaningVi** field rather than one free-text definition. She attaches **YouTube resources** to posts and watches them inside the article instead of losing her place to the browser, and she **exports** her library to Drive every week because it is the only copy.

2.3 Who this is not for

Saying no is part of the design. Learning Mocha is not a team wiki (no sharing, no permissions, no conflict resolution), not a general note-taking app (no images, no attachments, no handwriting, no web clipper), and not a flashcard app (no spaced repetition — **status** is a reading state, not a memory schedule). Each of those would have pulled the architecture towards accounts and a server, which is exactly what commitment 1 in §1 rules out.

2.4 What the user has to do, and what the app does for them

The user does this	So the app can do this
Writes <code>[[Spring Boot]]</code> inside a post	Resolves it to a real post, renders it as a tap target, records the reverse edge, and keeps it correct when either post is renamed
Sets a status and a few tags	Powers “continue reading” on the home screen, the related-posts list, tag browsing, and search filters
Pastes a YouTube URL into the body	Detects it and offers it as a playable resource card on the article
Adds a glossary term	Surfaces it as a chip on every article that mentions it, and makes it findable from search
Asks the assistant for a learning path	Reads only the parts of the library it explicitly queries, proposes a reviewable tree of posts with content, links, tags and glossary entries, and writes nothing until it is approved

3 Features

A tour of what the app does, grouped by what someone is trying to get done rather than by code package. The navigation graph declares fourteen destinations: five bottom-navigation tabs (Home, Browse, Search, AI, Settings) and nine pushed screens with typed arguments, each of which gets its turn below.

3.1 The knowledge base

Branches, folders and posts are three values of one `type` column in one self-referencing `nodes` table, so create, rename, move, reorder and delete are written once and behave the same at every level. §5 covers the schema.

Browse is the tree walker. A breadcrumb row starts at *Library* and adds a tappable crumb per level, so jumping back three folders is one tap; system back goes up one level. The `+` button offers *Branch*, *Folder* or *Post*, defaulting to *Branch* at the root and *Post* elsewhere, and choosing *Post* opens the editor with the current folder as the parent.

Each row has an overflow menu with *Rename*, *Move* and *Delete*. Dragging reorders among siblings, written when the drag ends; swiping left confirms, then deletes. *Move* is a computed list dialog rather than a drag — the deliberate limit is that dragging reorders but never re-parents — and `possibleParents` offers *Library* (*top level*) plus every branch and folder that is not inside the item being moved.

The rules the tree enforces are few, and most exist to keep something else honest:

Rule	Why
Only branches and folders take children	A child of a post would be invisible in Browse
A move that would cycle is refused	<code>TreeRules.wouldCreateCycle</code> walks the parent chain before the write, so a folder cannot land inside its own subtree
Two posts cannot share a title	Titles address posts everywhere — <code>[[wiki-links]]</code> , AI actions, lookup by name — so a duplicate makes every lookup arbitrary. Checked only when a title changes, so an old import with duplicates stays editable
Deleting a container deletes its contents	The foreign key cascades, the confirmation says so in words, and there is no undo

A first launch is not an empty screen: Room’s database-created callback seeds a *Getting Started* branch — a welcome post, a *How this library works* folder with two more, three tags, one per-post glossary term and one global one, wiki-linked so backlinks, related posts and the graph have something to show on day one. Seeding is tied to the database file being created, not to the library being empty: the seed text invites deletion, and re-growing it behind the reader’s back would be a bug.

3.2 Reading and writing

The **reader** renders Markdown through Markwon, entirely on-device. Above the article sit the title, the breadcrumb path, a star, a status chip and buttons for *Edit* and *Show in graph*; below it come glossary

chips, references, *Linked from* and *Related*, each hidden when empty, so a short post is a short screen. Opening a post bumps its `updatedAt`, so *Recent posts* reflects reading and not only editing.

[[Wiki-links]] are resolved at render time rather than stored. Before Markwon sees the text, each [[Title]] becomes a Markdown link on an internal scheme: `mocha://post/<id>` when the title matches a post, `mocha://missing/<title>` when it does not. A custom resolver intercepts both — the first navigates, the second reports “*Title* is not in your library yet”. An unresolved link is visible and inert rather than an error, because linking to a post you have not written is a normal way to plan.

The **editor** is a title field, a *Source/Preview* tab pair and a toolbar: bold, italic, heading, list, external link, wiki link, glossary term, add reference. Formatting buttons wrap the selection or insert a prefix at the start of the line. The wiki-link button wraps a selection in brackets, or with nothing selected lists every other post title and inserts the one picked. *Preview* runs the reader’s own rewrite, so links look exactly as they will when saved.

Every keystroke goes to the ViewModel and is rendered back from state, so rotation cannot lose a draft, and leaving with unsaved changes asks first. Tags are typed as a comma-separated list and deduplicated case-insensitively. References and glossary terms are staged in the draft: one transaction on save updates the node, rebuilds its outgoing link rows from the markdown, replaces its tag rows and upserts its terms.

Renaming a post is the interesting write. `KnowledgeSync.retitle` writes the new title, then rewrites [[old title]] in every post that links to it, in the same transaction, matching case-insensitively and tolerating padding inside the brackets. One routine backs the rename in Browse and in the editor. Without it an inbound link would degrade to “not created yet” and lose its link row on the linking post’s next save.

Deliberate limits: Markdown only — no images, no attachments, no rich-text toolbar — and the source pane is a plain text field with no syntax highlighting.

3.3 Connecting knowledge

Three deliberately different answers to “what else is this connected to” sit under each article. **Linked from** is exact: the posts whose markdown contains a resolved wiki-link to this one, read from the `links` table that saving maintains.

Related is inferred. It scores every other post and shows the top five: +2 per tag shared with this post; +2 for each post this one links out to; and +1 for each post that also links to one of those targets — co-citation, so two posts that both cite *Raft* surface for each other with no direct link between them. Posts already listed under *Linked from*, and the post itself, are excluded before scoring.

The **knowledge graph** draws the same data as a picture. Posts are dots sized by degree and filled by learning status, favourites accented; edges are wiki-links, and a chip switches between *Links only* and *Links + tags*, the second adding an edge between posts sharing a tag. From Home the graph is the whole library; from a post’s *Show in graph* it is that post plus its one-hop neighbourhood, which a *Whole library* chip widens. Two limits are made visible rather than hidden: past 250 posts only the most-connected are drawn and the caption says so (“ n most-connected of m posts”), and a tag on more than eight posts is drawn as a ring rather than a clique, because $O(k^2)$ grey edges would bury the wiki-links.

The layout is the project’s own code: `util/ForceLayout.java`, about 150 lines of Fruchterman–Reingold in framework-free Java — repulsion between every pair, attraction along edges, a cooling temperature, sunflower-spiral seeding so isolated nodes start spread out instead of piled on one radius, and a fixed seed so the same library draws the same shape between launches. It returns a flat `float[]` in a fixed square world that `GraphView` fits to whatever screen it gets, so rotation never re-runs the layout. Layout runs on `Dispatchers.Default`; the view handles drag-to-pan, pinch-to-zoom, double-tap-to-fit, tap-to-select and tap-again-to-open, and hides node labels when the dots are packed too tightly to read.

3.4 Learning metadata

A post carries four pieces of state beyond its text, each earning its place by feeding another screen. **Status** is *None*, *Reading*, *In progress* or *Finished*, set from a chip in the reader or the editor. It is a reading state, not a memory schedule — there is no spaced repetition — and it drives *Continue reading* on Home (posts in *Reading* or *In progress*, most recently touched first) and two search filters. **Favourite** is the star in the reader header, filling the Home section, the Favorites screen, the favourites search filter and the accent ring in the graph. **Tags** are stored as shared rows; they appear as chips under the reader title, tapping one opens a screen of the posts carrying it, and a **Tags** screen lists every tag alphabetically with a post count and a filter box.

References are external material: a kind (YouTube, Article, Book, Other), an optional title and a URL, attached from the editor and shown as cards under the article captioned with the link’s host. A

post’s reference list is the union of the rows someone added on purpose and the YouTube URLs sitting in the markdown right now, detected at read time and deduplicated by video id. The derived half is not stored, which stops a re-save from deleting a reference the user or the assistant added by hand; the trade is that a derived card is removed by editing the markdown rather than by a close button, and the editor marks it accordingly.

Tapping a YouTube card plays it in a bottom sheet without leaving the article: a locked-down `WebView` loading the `IFrame` embed against `youtube-nocookie.com`, file and content access off, navigation restricted to an allow-list of YouTube hosts and everything else handed to the browser. An *Open in YouTube* button is always present, and a link with no eleven-character video id — a channel or a playlist — opens externally rather than pretending to play.

3.5 Dictionary

A glossary entry is a term, an English definition and a Vietnamese meaning, in a dedicated `meaningVi` field rather than one free-text blob, because a reader of English-language material wants both and wants them separable. An entry belongs to one post or is global.

While reading, the reader shows a chip for every entry attached to this post *or* whose term appears in the body, matched case-insensitively, so a global term surfaces only on the articles that use it. Tapping a chip opens a bottom sheet rather than a dialog, so the definition appears without covering the paragraph the word was read in. An *Open dictionary* link sits beside the chips for when a definition is not enough.

The **Dictionary** screen is the glossary as a whole, with full editing: a search box filtering on term, definition and Vietnamese meaning at once; scope chips for *All*, *Global* and *Per post*; rows showing the term, its definition and its origin (“Global term”, or “From *Post title*”). The + button adds a global term, since anything created here belongs to no single post. A row tap edits; the row menu edits, jumps to the owning post, or deletes with an undo snackbar. Adding a term that already exists in the same scope updates that row instead of stacking a duplicate chip in the reader.

3.6 Finding things

Search is one text box, debounced at 300 ms, plus five filter chips. The box drives an FTS4 index over post titles and content; because `posts_fts` is content-backed by `nodes`, Room’s triggers keep it in step and no application code writes to the index. The typed text becomes a MATCH query with punctuation stripped and each token given a prefix star, so “dist sys” matches *Distributed Systems* while a stray quote cannot break the query.

The result list merges five passes and de-duplicates: FTS hits on title and body, a title substring pass (so branches and folders match too), posts carrying a matching tag, glossary entries, and references. Posts sort first, then title-prefix matches, then alphabetically, capped at fifty; a post matching only in its body carries an excerpt around the first match, snapped to word boundaries, so the row says why it is there. The chips are *Posts* and *Branches* (mutually exclusive), *Favorites*, and *Reading/Finished* (also mutually exclusive); set with nothing typed, they list the library on their own. Glossary and reference hits appear only when no filter is active, having no type, status or star to filter on. Tapping a result goes where it lives: a post opens the reader, a branch or folder Browse at that node, a glossary hit its owning post or the dictionary, and a reference the post that cites it.

Home covers the parts you should not have to search for: a greeting chosen by the hour, the post count, four shortcut tiles (graph, favourites, dictionary, tags), a horizontal *Continue reading* strip, *Recent posts*, *Favorites*, and a branch list opening Browse at the branch tapped.

3.7 The AI assistant

The **AI** tab lists chat sessions; a conversation carries four mode chips — *Answer*, *Suggest*, *Modify* and *Organize* — deciding the system prompt and what the model may return. The assistant reads the library only through read-only context queries, so it is never uploaded wholesale, and it cannot write: proposals arrive as structured actions, and the **Review changes** screen shows one row per action to preview, edit, relocate or deselect before the approved subset is applied. §6 covers the modes, the protocol, the validation rules and the transaction behaviour in full.

3.8 Backup, settings and theming

Export writes the whole library through the system file picker, into a file named for the current date with a `.mocha.json` suffix: one versioned JSON document holding nodes, links, tags, post-tag pairs, glossary

entries and references. Chat history is deliberately not in it — a transcript of questions is not part of the knowledge base, and exporting it would put messages into a file people share.

Import takes a file from the same picker, reads it without writing anything, then asks: this file holds n posts — *Merge* into the library, or *Replace* everything? Restore always reassigns primary keys and rewrites every foreign key against the new ids, so a merge cannot collide with rows already present and a replace cannot inherit stale ones. Nodes go in parents-first; a node whose parent is missing lands at the top level rather than disappearing, and a per-post glossary entry whose post is missing is dropped. The search index needs no attention, because it is content-backed.

Because the device is the only copy, the app nags. A *Remind me weekly* toggle posts a notification on the next cold start once a week has passed since the last export — or since first launch, tracked separately so someone who has never exported is not told they have. The notification permission is requested when the toggle is switched on, and the check runs at startup rather than on a schedule, avoiding a background scheduler for a message that only has to be seen when the app is next opened.

Settings also holds the theme (*Follow system*, *Light*, *Dark*), applied through `AppCompatDelegate` and kept in `SharedPreferences` so it can be read synchronously in `Application.onCreate`, before any view inflates and therefore without a flash of the wrong palette. Below it sits the **AI gateway address**, with *Save*, *Reset* and *Test connection*; the address is normalised to end in a slash, defaults to the emulator loopback, and the test reports plainly — “Connected to the gateway”, or “Could not reach the gateway. Your library still works.” A privacy paragraph beneath states what leaves the device and what never does.

3.9 Offline and degraded states

Every screen except the assistant works with the network switched off, and works identically — none of them makes a network call at all. That follows from the local-first commitment in §1; it is not a fallback path that has to be maintained separately.

When the **gateway is unreachable**, both chat screens ping its health endpoint on entry and show a banner with a retry button, the send button disabled, worded reassuringly: “AI unavailable — your library still works.” Nothing is written and nothing is queued, and a reply mid-stream when the connection dies leaves no trace either (§6). A YouTube sheet with no network swaps the player for “Video unavailable offline. Open it in YouTube to try again.”

Empty and error states are one mechanism rather than thirteen. `ui/common/ListState` is a five-value enum — `LOADING`, `CONTENT`, `EMPTY`, `ERROR`, `OFFLINE` — and `ListStateBinder` renders it into a shared overlay of spinner, message and retry button. All thirteen list-bearing screens bind it, each supplying its own wording, so an empty state says something useful instead of “no data”: the graph says “Nothing to draw yet. Write a few posts and connect them with `[[wiki-links]]`”.

One honest gap: the `OFFLINE` value is defined in the enum and handled in the binder, but no `ViewModel` emits it. The chat screens express unreachability through their own banner and a disabled send button instead, which is the better fit — an offline chat should still show its history rather than be replaced by a full-screen message — so that branch of the binder is unused code today rather than a state the user can reach.

4 Application architecture

4.1 The system in one picture

Learning Mocha is an Android application plus a small Express process, and the boundary between them is not the usual client/server split. The device owns the data; the server owns a secret. Nothing in `backend/` has a database, a session table or a disk write, and no part of the library reaches it except text a user has deliberately typed into a chat message. Figure 1 is the whole system.

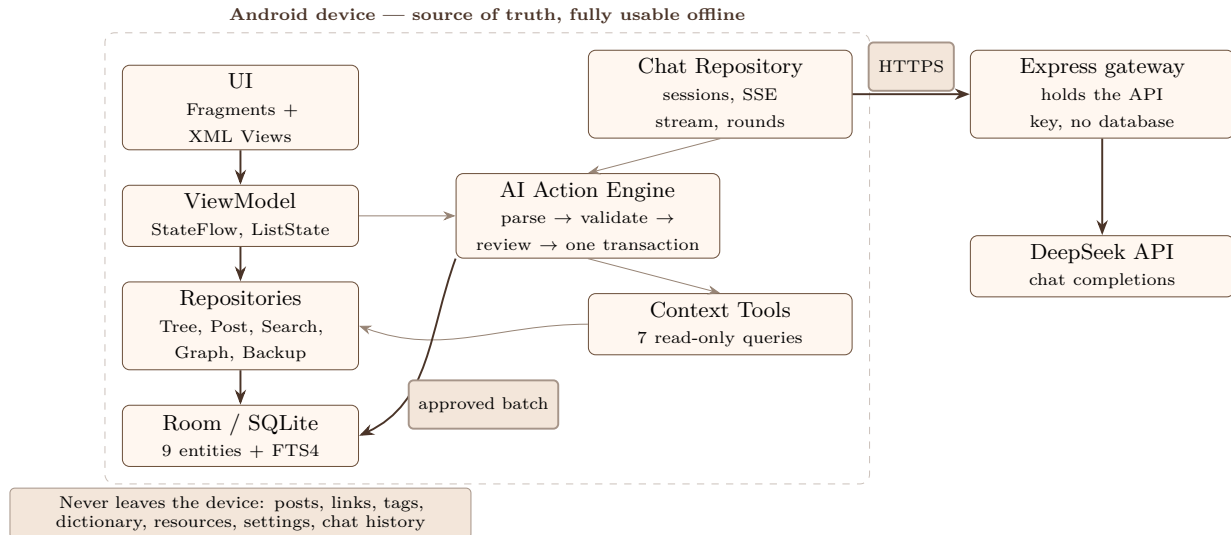


Figure 1: Overall architecture. Knowledge lives only in Room on the device; the backend is a stateless key-holding proxy, and every AI-originated write passes through validation and an explicit user review before a single Room transaction applies it.

The network is touched in exactly three places, all behind the AI tab: `GET /v1/health`, so the chat screen can say “AI unavailable” before the user commits to a question; `POST /v1/chat`; and `POST /v1/chat/stream`, the same body answered as Server-Sent Events. There is no fourth. Home, Browse, the reader, the editor, search, tags, the glossary, the graph and export/import never open a socket, which is why offline is not a mode here — it is the normal case, and the AI tab is the exception that has to cope with a network. The app holds no API key (`DEEPSEEK_API_KEY` lives only in the gateway’s `.env`), and Android’s Auto Backup is switched off in the manifest so the OS does not upload the Room file either.

4.2 Layers on the device

Inside the app the structure is conventional MVVM with a repository layer, deliberately: the interesting parts of this project are the knowledge model and the action protocol, and neither is helped by an inventive architecture underneath. Rule 3 of `AGENTS.md` is the constraint the codebase is held to — “*Layers: Fragment → ViewModel → Repository → DAO. No DB/network calls in UI code.*”

Data flows one way. A repository exposes Room Flows; a ViewModel combines them into exactly one immutable UI-state data class published as a `StateFlow`; a Fragment collects that one flow and renders it. Events travel back as plain method calls on the ViewModel.

Listing 1: Package layout under `app/src/main/java/com/cs426/learningmocha/`

<code>LearningMochaApp.kt</code>	Application: 8 lazy singletons, theme, backup reminder
<code>MainActivity.kt</code>	the only Activity: NavHost + bottom navigation
<code>ui/</code>	14 Fragments + 1 bottom sheet, adapters, list-state views
<code>viewModel/</code>	15 ViewModels, one UiState data class each
<code>data/local/</code>	entity/ (9 entities), dao/ (Node, Knowledge, Chat), AppDatabase, KnowledgeSync, SeedData, InlineResources
<code>data/repo/</code>	Tree, Post, Search, Graph data/prefs/ SettingsStore
<code>ai/protocol/</code>	Envelope, ActionParser, ActionValidator
<code>ai/engine/</code>	ActionExecutor, ContextTools, KbIndex
<code>ai/chat/</code>	ChatRepository
<code>net/</code>	ApiClient, MochaApi, SseChatClient, SseFrames, ChatDtos
<code>backup/</code>	BackupRepository, BackupSnapshot, BackupReminder
<code>util/</code>	8 pure-Java classes (below)

Browse, end to end: `BrowseFragment` collects `BrowseViewModel.uiState`, which flat-maps the current folder id over `TreeRepository.observeChildren(parentId)`, one indexed `SELECT` in `NodeDao`. A drag ends in `ItemTouchHelper.clearView`, which calls `viewModel.persistOrder(ids) → TreeRepository.reorder`, which writes the new `orderIndex` values inside `db.withTransaction`. Room re-emits the query and the list re-renders; the Fragment never mutates its own data.

Rules live in the repository, not the ViewModel and not the DAO. Renaming a post is the clearest case: it refuses a title another post already holds, rewrites every `[[old title]]` in every post that links to it, and

re-derives those posts' link rows — in one transaction, so an interrupted rename cannot leave dangling links. There is one honest exception to the layering rule: `TagsViewModel` reads `KnowledgeDao` directly, because neither `tags` nor `post_tags` exposes a `Flow` and the tag index needs a per-tag count no repository returns. It is a commented shortcut, not a pattern.

4.3 Why some of it is Java

Rule 4 of `AGENTS.md`: Kotlin for Android-facing code, Java only for framework-free utilities in `util/`. The split is chosen so the language boundary and the test boundary are the same line — a class that cannot import `android.*` is a class that can be unit-tested on the JVM in milliseconds.

Java class in <code>util/</code>	Framework-free job
<code>MarkdownLinkParser</code>	Extract <code>[[wiki-links]]</code> and YouTube URLs; rewrite a link title on rename
<code>FtsQueryBuilder</code>	Sanitise typed input into a valid FTS4 <code>MATCH</code> expression, or <code>null</code>
<code>TreeRules</code>	<code>wouldCreateCycle</code> — stops a move turning the tree into a graph
<code>ExportJsonWriter</code>	Stream the <code>.mocha.json</code> envelope out and read it back, with format
<code>ImportJsonReader</code>	and version checks
<code>TextDiff</code>	Line diff behind the review screen's "what would change"
<code>StreamingAnswerExtractor</code>	Pull readable prose from a JSON envelope still arriving
<code>ForceLayout</code>	Fruchterman–Reingold on primitive arrays, for the knowledge graph

Each has a JVM unit test under `app/src/test/`, and the proportion works out at about 970 lines of Java against 9,500 of Kotlin.

4.4 Threading, lifecycle, and what survives what

All asynchronous work is coroutines — no `AsyncTask`, no `Executor`, no callbacks in the data layer. Room's suspend and `Flow` DAOs already dispatch onto the database's own executor, so `Dispatchers.IO` appears in only three places in the whole app (file streams in `BackupRepository`, the SSE read loop in `SseChatClient`, the snapshot read in `GraphViewModel`) and `Dispatchers.Default` once, for the force-directed layout. Everything else needs no thread hop, which is itself an argument for Room.

Collection is tied to the view rather than the `Fragment`: every screen collects inside a `repeatOnLifecycle(STARTED)` block on the view lifecycle owner, and every `uiState` is published through `stateIn` with `SharingStarted.WhileSubscribed(5_000)`, so a rotation does not tear down and rebuild the underlying query. Views come from `viewBinding` held in a nullable field cleared in `onDestroyView`.

That leaves two losses to handle explicitly. A **configuration change** is survived by the `ViewModel`, so the current folder, an in-flight AI stream and a half-written editor draft all outlive a rotation with nothing written to disk. **Process death** is not: seven `ViewModels` take a `SavedStateHandle`, through which navigation arguments (`postId`, `tagId`, `sessionId`, `messageId`) arrive for free, and into which `BrowseViewModel` additionally writes the folder being viewed. Anything more valuable is already in Room by then — an AI batch awaiting approval, for instance, lives as `actionsJson` on its `chat_messages` row, so it is still reviewable after a cold start.

4.5 Navigation

One `MainActivity`, one `NavHostFragment`, and `nav_graph.xml` for the rest. Five destinations are top-level and carry the `bottom_nav_menu` ids, which is all `setupWithNavController` needs: **Home**, **Browse**, **Search**, **AI**, **Settings**. Nine more sit on the back stack — post reader (`postId`), post editor (`postId`, `parentId`), favourites, dictionary, tag detail (`tagId`), knowledge graph (`focusPostId`, 0 = whole library), tags, conversation (`sessionId`) and review changes (`messageId`) — and `MainActivity` hides the bottom bar for them with one `addOnDestinationChangedListener` rather than a per-screen flag.

Bottom navigation

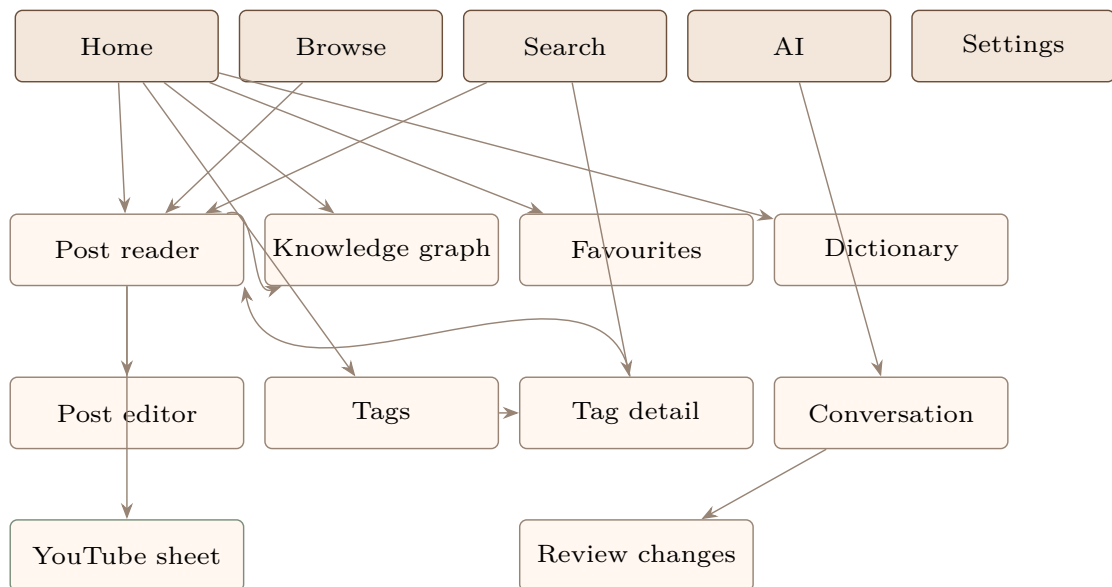


Figure 2: Navigation graph. One `MainActivity` hosts five bottom-navigation tabs; nine further destinations sit on the back stack (the tab bar hides itself for them), and the YouTube player is a bottom sheet rather than a destination so playback stops with the sheet. The reader, editor, favourites, dictionary, tags, tag detail, graph, conversation and review destinations are all declared as global actions, so any screen can open them without a dedicated edge in the graph.

All nine are reached through *global* actions. That matters more than it sounds: the reader is opened from Home, Browse, Search, tag detail, the graph and the review screen, and per-screen actions would have meant six near-identical edges and six chances to forget a transition. Only reader → editor is local, because only the reader has an Edit button; every action shares one 180 ms animation set from `res/anim/nav_*`.

Two deliberate non-destinations: the YouTube player is a `BottomSheetDialogFragment`, so dismissing the sheet destroys the `WebView` and stops playback with no lifecycle code of our own, and the create/rename/move/delete dialogs are `MaterialAlertDialogBuilder` calls, because none deserves a back-stack entry. Window insets are handled once, in `MainActivity`, which pads the root view for system bars, cutout and IME under `enableEdgeToEdge()`.

4.6 The invariant

No AI-originated write reaches Room outside `ActionExecutor`

The model has no DAO, no repository and no database handle. It reads the library only through seven named read-only queries in `ai/engine/ContextTools`, and changes it only by emitting a batch of structured actions, which is parsed, validated against live data, shown on a review screen where each item can be deselected, and then applied by `ai/engine/ActionExecutor` in a single `db.withTransaction`. The codebase has exactly one call site for that executor: `ReviewChangesViewModel`, on the approve button.

The mechanism — envelope format, validator, two-round context protocol, review screen, single-level undo — is §6. The architectural consequence is the part that belongs here: the AI is a peer of the UI layer, not a layer beneath it, and it reaches the database through a narrower door than the user's own screens do.

5 Data model and storage

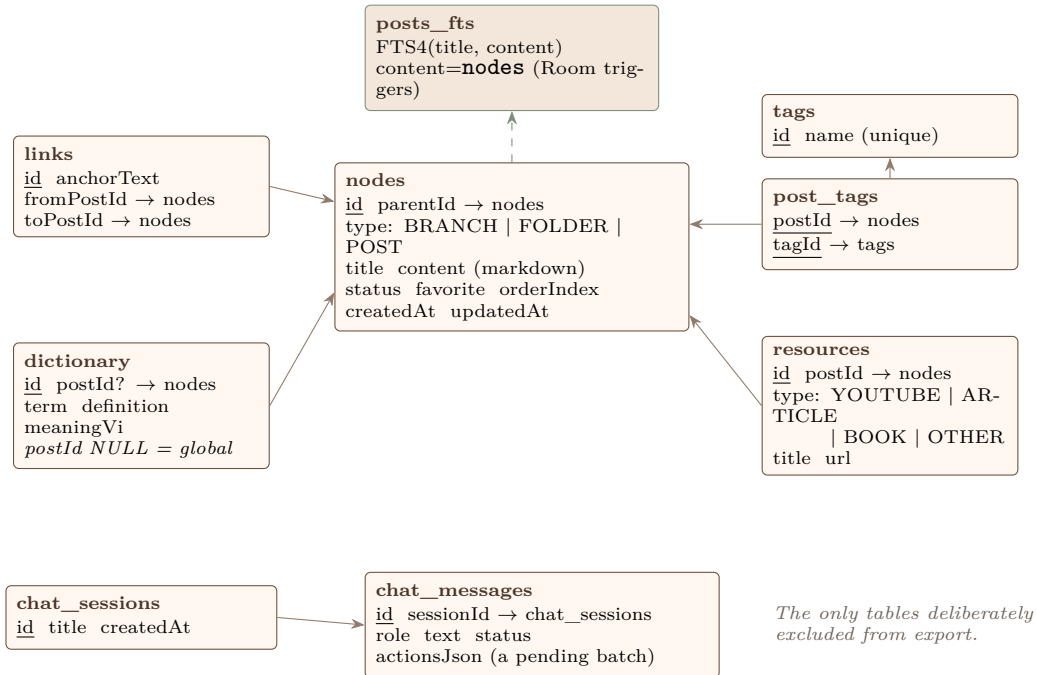
5.1 Why a database and not files

The obvious alternative was a folder of Markdown files plus a small index — what most note apps do, and it would have made backup trivial. It was rejected for four properties the app actually uses. **Transactions:** renaming a post rewrites the title, rewrites `[[old title]]` inside every post that links to it, and rebuilds

those posts' link rows, which on a filesystem is n independent writes with no way to undo the first six when the seventh fails. **Observable queries:** every screen is a **Flow** from a DAO, so when an AI batch commits, Home, Browse and the reader update themselves and nothing has to know who wrote what. **Migrations:** the schema changed twice during development with an installed build on a real phone each time, and `MigrationTestHelper` makes that testable. **Full-text search:** SQLite ships FTS, a file tree ships `grep`. The cost is one opaque `learning_mocha.db` the user cannot open in a text editor, which is why export is a first-class screen rather than an afterthought (§3).

5.2 One table for the tree

Branches, folders and posts are all rows in `nodes`, distinguished by a `type` column and parented by a nullable self-referencing `parentId`. Figure 3 is the whole schema.



Every arrow is a real **ON DELETE CASCADE** foreign key, so deleting a branch removes its subtree, links, tags, glossary entries and resources in one statement. Backlinks are a reverse query on `links`, never a stored second copy, and `posts_fts` is content-backed so Room's generated triggers keep the index in step with `nodes` automatically.

Figure 3: The Room schema (database version 3). Branches, folders and posts share one self-referencing `nodes` table, which makes every tree operation identical regardless of node type.

The payoff is that every tree operation is written once and is type-independent. `create`, `rename`, `move`, `reorder` and `delete` in `TreeRepository` never branch on node type; `observeChildren` is one indexed `SELECT` returning a mixed list already in `orderIndex` order; `breadcrumbs` is a `parentId` walk that does not care what it walks through; drag-to-reorder works on a heterogeneous list without a single `when (type)`. Deleting a branch is `DELETE FROM nodes WHERE id = ?`, and **ON DELETE CASCADE** takes the subtree, its links, tag rows, glossary entries and resources with it — one statement, no orphans.

The cost is real. `content`, `status` and `favorite` are meaningless for containers and sit there as `NULL`, `'NONE'` and `0`, and nothing at the database level stops a folder from having body text — that rule lives in the repositories, not in a constraint. For a schema this size the trade is clearly worth it: the alternative is three tables, three sets of CRUD and a union query every time Browse needs a folder's children.

`nodes` carries four indices — `parentId` (children), `title` (link resolution and duplicate checks), `updatedAt` (Home's "recent" and "continue reading") and `type` — and the three enums are stored as names through `NodeConverters`, not ordinals, so reordering an enum in Kotlin cannot silently reinterpret existing rows.

5.3 The satellite tables

Table	Why it exists
<code>links</code>	One row per resolved <code>[[wiki-link]]</code> . Owned entirely by <code>KnowledgeSync.reindex</code> , which deletes a post's outgoing rows and re-derives them from its Markdown on every save, so the table cannot disagree with the text. Both columns are indexed because both directions are queried.
<code>tags / post_tags</code>	A unique-by-name tag list and a join table keyed on the <code>(postId, tagId)</code> pair. Insertion uses <code>OnConflictStrategy.IGNORE</code> with a case-insensitive lookup, so “Raft” and “raft” are one tag.
<code>dictionary</code>	<code>term</code> , English <code>definition</code> , Vietnamese <code>meaningVi</code> . The nullable <code>postId</code> is the feature: <code>NULL</code> means global, a value scopes the term to one post. Uniqueness per scope is an upsert in <code>KnowledgeSync.addTerm</code> , not a constraint.
<code>resources</code>	Explicitly added references (<code>YOUTUBE</code> , <code>ARTICLE</code> , <code>BOOK</code> , <code>OTHER</code>). YouTube URLs that merely appear in the Markdown are <i>not</i> stored — they are derived at read time by <code>InlineResources.merge</code> with <code>id = 0</code> so the UI can tell them apart. Rebuilding this table from the body on save used to delete references added by hand.
<code>chat_sessions / chat_messages</code>	Local chat history. A message carries <code>role</code> , <code>text</code> , a <code>status</code> (<code>ok</code> , <code>error</code> , <code>pending_review</code> , <code>applied</code> , <code>discarded</code>) and a nullable <code>actionsJson</code> holding a proposed batch — which is what lets a pending review survive a cold start.

There is no `backlinks` table, and that is the point: a backlink is `SELECT ...FROM links WHERE toPostId = :postId` joined back to `nodes` — a reverse read of data that already exists. Storing the reverse edge would double the write path and create a second thing that can be wrong; deriving it makes backlinks correct by construction, and `index_links_toPostId` makes it cheap. Related posts are derived the same way, by scoring shared tags and shared link targets in `PostRepository.related`.

5.4 Full-text search: `posts_fts`

Search is a real FTS virtual table, not a `LIKE` scan. It is **FTS4**, declared as `@Fts4(contentEntity = Node::class)` — Room 2.6 has no `@Fts5` annotation, and adopting FTS5 would have meant hand-written `CREATE VIRTUAL TABLE` and hand-written triggers for no benefit this app can measure.

`contentEntity` matters more than the version: it makes the index *content-backed*, storing no copy of the text, and Room generates four triggers on `nodes` (before update, before delete, after update, after insert) that keep `docid` in step with `nodes.rowid`. Nothing in the app ever writes to `posts_fts` — one query reads it, none writes it — and searching joins back for real rows: `SELECT nodes.* FROM posts_fts JOIN nodes ON nodes.rowid = posts_fts.rowid WHERE posts_fts MATCH :query`. Restoring a backup, applying an AI batch and seeding all get a correct index for free, because they all write to `nodes`; the index is likewise absent from the backup file, and `MIGRATION_1_2` ends with `INSERT INTO posts_fts(posts_fts) VALUES('rebuild')` for the same reason.

Two honest limitations. The index covers every row in `nodes`, so branch and folder titles are searchable too — treated as a feature here, since `SearchRepository` labels container hits and routes them to Browse rather than the reader. And FTS4 has no `bm25` ranking (that is FTS5), so ordering happens in Kotlin: posts first, then title prefix matches, then alphabetical.

5.5 Migrations and schema history

The database is at **version 3** with `exportSchema = true`, and both migrations are hand-written in `AppDatabase.kt`. `MIGRATION_1_2` adds Phase 2's entire knowledge layer — `links`, `tags`, `post_tags`, `dictionary`, `resources` with their indices and foreign keys, the `posts_fts` virtual table, the four content-sync triggers, and a `'rebuild'` so existing posts become searchable immediately rather than after their next edit. `MIGRATION_2_3` adds Phase 3's `chat_sessions`, `chat_messages` and `index_chat_messages_sessionId`.

`fallbackToDestructiveMigration()` is not called anywhere. It is one line, it makes the migration warning disappear, and it deletes the user's library; for an app whose central promise is that the device holds the only copy, its absence is a decision rather than an oversight.

The exported schemas live in `app/schemas/com.cs426.learningmocha.data.local.AppDatabase/` and are committed; `build.gradle` adds that directory to `androidTest.assets.srcDirs`, which is what lets `MigrationTestHelper` build an old database from a fixture. Two instrumented tests in

`app/src/androidTest/java/com/cs426/learningmocha/data/local/MigrationTest.kt` exercise the real upgrade paths against real rows. `migrates1To3KeepingPosts` writes a `v1` file by hand with a branch and a post in it — `v1` predates `exportSchema`, so there is no `1.json` — runs both migrations through a real `Room.databaseBuilder`, and asserts that the post survived with its status and favourite flag, that `posts_fts MATCH 'spring'` finds it (the rebuild worked), and that the chat tables exist and are empty. `migrates2To3AddingChatTables` builds `v2` from `2.json`, inserts a post, a tag and a tag link, then calls `runMigrationsAndValidate(..., 3, true, MIGRATION_2_3)`, which additionally validates the resulting schema against `3.json` column by column. The two tests are shaped differently only because `exportSchema` was turned on in Phase 2 rather than Phase 0 — the one thing about this area worth doing differently next time.

5.6 Settings

Preferences do not live in Room. `data/prefs/SettingsStore` wraps one `SharedPreferences` file with five keys: theme mode, gateway URL, whether backup reminders are on, the timestamp of the last real export, and a separate reminder clock (kept apart so a user who has never exported is never told they have).

It is `SharedPreferences` rather than `DataStore` or a table because of startup ordering. Applying the theme means calling `AppCompatDelegate.setDefaultNightMode` from `Application.onCreate`, which must happen before the first view inflates or the app flashes the wrong theme — so that read has to be synchronous. `DataStore`'s API is suspending by design, and reading a Room table there would open the database on the main thread during cold start, the opposite of what `LearningMochaApp` is arranged to avoid: every repository is a `by lazy` and the database file is not touched until a screen asks for data.

5.7 The backup file

`ExportJsonWriter` streams a single `.mocha.json` out through the Storage Access Framework. The envelope is `{format: "mocha.backup", version: 1, exportedAt, nodes[], links[], tags[], postTags[], dictionary[], resources[]}` — the six tables that are the user's knowledge, and nothing else. `posts_fts` is excluded because it is derived and Room's triggers rebuild it during the import writes. **Chat history is never exported:** a backup is a file people put on Drive or mail to themselves, and transcripts are the most sensitive text in the app, so `chat_sessions` and `chat_messages` are left out as a privacy decision recorded in `BackupSnapshot`'s `KDoc`.

Import skips unknown fields, so a file from a newer minor build still loads, but rejects one whose `format` is not `mocha.backup` or whose `version` is newer than the app understands. The user picks *merge* or *replace*, and the restore runs in one transaction.

What makes merge safe is that **ids from the file are never trusted**. Nodes are inserted parents-first (a topological pass with a cycle guard), each insert's new primary key is recorded in an old-id → new-id map, and every foreign key — `parentId`, `fromPostId`, `toPostId`, `postId`, `tagId` — is rewritten through that map before its row is written. Tags are matched by name instead, so importing into a library that already has a `consensus` tag reuses it rather than duplicating it. The edge cases are decided explicitly: a node whose parent is missing is restored at the root rather than dropped, because losing a subtree silently is worse than misplacing one; a glossary entry whose owning post is missing is dropped; a global term stays global. Without the remapping, merging into a non-empty library would collide on primary keys and replacing would inherit ids that no longer mean anything.

6 The AI learning assistant

6.1 What makes it more than a chatbot

The assistant can *act* on the user's library — create a branch of posts, rewrite an article, move a folder, attach a glossary term — and is structurally incapable of doing any of it by itself.

Its entire output is one JSON object: no database handle, no file access, no tool-calling API, no way to name a row id — it addresses everything by title. Proposed operations are parsed on the device, validated against the live database, rendered as a checklist, and applied only on an explicit tap inside one Room transaction. Figure 4 is that write path on one page.

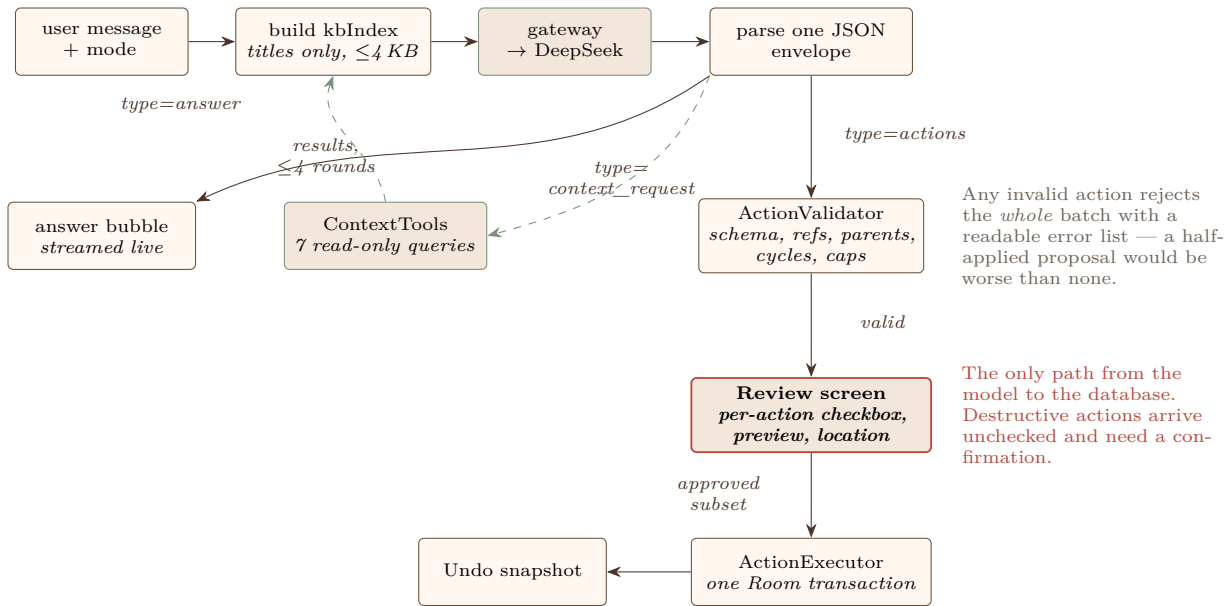


Figure 4: The AI write path. The model never sees the database and never writes to it: it emits a JSON envelope, the device validates it against live data, the user approves individual actions, and one Room transaction applies them all-or-nothing.

6.2 Four modes

The mode travels with every request as a plain string; anything outside the set is rejected with HTTP 400. It selects a short brief appended to the system prompt.

Mode	The brief appended to the system prompt
Answer	Answer conversationally, propose nothing, use <code>context_request</code> to look things up.
Suggest	Recommend posts, paths, links, resources or terms as a batch; “prefer small, high-value suggestions”.
Modify	Propose concrete changes — create, move, edit, link, tag — with “genuinely educational markdown”. The mode that generates whole articles and learning paths.
Organize	Analyse a branch and propose restructuring: moves, new folders, links, duplicate detection.

The action-protocol block goes out in *all four* modes; only the brief differs, so the Answer-mode guard lives in `ChatRepository` rather than the prompt — a prompt instruction is a request, not a constraint. Actions proposed in Answer mode never become a reviewable row; the summary is shown as prose plus “*switch to Suggest or Modify to review them*”.

6.3 The gateway

The app never talks to DeepSeek. It talks to a stateless Express process of about 260 lines that holds the key, builds the request, normalises errors and relays the stream. It has no database: every request carries its own history. Three routes:

- GET `/v1/health` → `{ok, model, streaming}`: `ok` is whether the key is configured, so a keyless gateway reports itself unusable up front, and `streaming` saves a probe for a 404.
- POST `/v1/chat` — buffered. Body `{mode, messages[], kbIndex, toolResults?, response_format?, max_tokens?, stream?}`, response `{reply, usage}`.
- POST `/v1/chat/stream` — same body, answered as Server-Sent Events.

Upstream it always asks for `response_format: json_object` and `max_tokens` of 8192, because DeepSeek otherwise stops at its default length and returns a truncated envelope.

Listing 2: A request, its reply, and a normalised error. The whole answer is the `reply` string, itself a JSON envelope parsed on the device.


```

POST /v1/chat
{ "mode": "modify",
  "messages": [{ "role": "user", "content": "Give me a path into consensus algorithms" }],
  "kbIndex": "- Distributed Systems (branch)\n - Paxos (post) [reading]\n",
  "toolResults": null }

200 OK
{ "reply": "{ \"type\": \"actions\", \"summary\": \"Create a Consensus path\", ... }",
  "usage": { "prompt_tokens": 1841, "completion_tokens": 962 } }

502
{ "error": "The reply was cut off before it finished. Try asking for less at once.",
  "retryable": true }

```

The key is read from `DEEPSEEK_API_KEY` in the gateway’s `.env`, in neither git nor the build; the APK holds only the gateway address, an editable Settings field.

Every failure reaches the app as `{error, retryable}`: missing key → 503 non-retryable; unknown mode or empty `messages` → 400 non-retryable; upstream 429 → 429; other upstream failures → 502 carrying DeepSeek’s message; timeout → 504 (60 s buffered, 120 s streaming); `finish_reason == "length"` or a blank reply → 502 retryable. Streaming errors arrive as a frame, headers being already flushed, and a stream ending without `done` fails.

6.4 Choosing what the model may see

Sending the whole knowledge base would be expensive, would break as the library grew, and would ship every private note off the device on every message. The model gets a map and must ask for the territory.

`KbIndex.build` walks the tree depth-first in sibling order, one indented line per node: title, type, and reading status where it is not `NONE`. No bodies, tags, ids or timestamps, and a hard cap of `MAX_CHARS = 4000` characters, truncated mid-line.

Listing 3: The compact KB index — the only context sent unprompted.

```

- Distributed Systems (branch)
  - Consensus (folder)
    - Paxos (post) [reading]
    - Raft (post) [in_progress]

```

If it needs more it replies with a `context_request`; the device runs the queries locally and returns the results. Seven read-only operations exist in `ContextTools`:

Operation	Argument	What comes back
<code>search_posts</code>	<code>query</code>	Up to 10 hits as <code>KIND: Title</code> , from the same full-text search the Search tab uses.
<code>get_post</code>	<code>title</code>	Title, tag names and the full Markdown body. The only operation that returns article text.
<code>list_children</code>	<code>parentTitle</code>	That container’s children as <code>type: title</code> ; blank returns the root branches.
<code>get_backlinks</code>	<code>title</code>	Titles of posts whose bodies link to this one.
<code>search_dictionary</code>	<code>query</code>	Up to 15 glossary rows as <code>term: definition</code> ; wildcards are stripped from the fragment.
<code>get_tags</code>	—	Every tag name, one per line.
<code>get_related</code>	<code>title</code>	Up to five from <code>PostRepository.related</code> (shared tags and link targets, no backlinks).

Three caps apply: **8 queries** per request, each result truncated to **2 000 characters**, and **repeat(4)** in `ChatRepository` — **four model calls per user message**, at most three of them context requests. A `context_request` on the fourth is refused with a retryable “*Could not gather enough context*” rather than looping.

Results come back as an ordinary `user` turn labelled “*CONTEXT TOOL RESULTS ... (read-only, produced on-device)*” — not the OpenAI `role: "tool"` turn, which DeepSeek rejects with HTTP 400 unless it carries a `tool_call_id`. Each reply carries “*Shared with AI: n notes*”, *n* counting queries run.

6.5 The action protocol

The reply is one JSON object and nothing else, of three envelope types: **answer** carries **text** and renders as a bubble; **context_request** carries **queries** and triggers the round trip above; **actions** carries a **summary** and the action list for the review screen.

ActionParser is forgiving about packaging and strict about content: it strips a code fence, takes the substring from the first { to the last }, and Gson-parses it. A blank or missing **type** is an error; prose that is no envelope is wrapped as an **answer** by **parseOrAnswer**, so forgetting the format is never fatal.

Listing 4: An **actions** envelope; **ref** names an item created earlier in the same batch.

```
{ "type": "actions",
  "summary": "Create a Consensus path with 4 posts",
  "actions": [
    {"op": "create_branch", "title": "Distributed Systems", "ref": "b1"},
    {"op": "create_folder", "parentRef": "b1", "title": "Consensus", "ref": "f1"},
    {"op": "create_post", "parentRef": "f1", "title": "Raft", "ref": "p1",
      "content": "Raft is a consensus algorithm ... see [[Paxos]].",
      "tags": ["consensus"], "status": "READING"},
    {"op": "create_link", "fromRef": "p1", "toTitle": "Paxos"},
    {"op": "add_dictionary_entry", "postRef": "p1", "term": "quorum",
      "definition": "A majority of nodes.", "meaningVi": "..."} ] }
```

Fourteen operations are implemented, listed below as **ActionValidator.knownOps**; **backend/prompts.js** documents the same set, each side commented that changing one means changing the other.

Operation	Key fields	Effect when applied
create_branch	title , ref	New root container.
create_folder	parentRef or parentTitle , title , ref	New container under an existing or in-batch parent.
create_post	+ content , tags , status	New article; tags created on demand, status defaults to READING .
update_post	postTitle , title , content , tags , status	Rewrites body, title, tag set and/or status.
move_post	postTitle , newParentTitle	Re-parents a post.
delete_post	postTitle	Deletes the post and everything hanging off it.
create_link	fromRef or fromTitle , toTitle	Appends [[Target]] to the source body, which creates the edge.
remove_link	fromTitle , toTitle	Removes that wiki-link from the body.
add_tag , remove_tag	postTitle , tag	One tag on one post.
set_status	postTitle , status	READING , IN_PROGRESS or FINISHED .
set_favorite	postTitle , favorite	Stars or unstars a post.
add_resource	postTitle , type , title , url	Attaches a YOUTUBE , ARTICLE , BOOK or OTHER reference.
add_dictionary_entry	postTitle (optional), term , definition , meaningVi	Adds or updates a glossary term, post-scoped or global.

Nothing here is vendor-specific: no function-calling schema, no tool role, no provider SDK — a documented JSON contract over ordinary chat completions. Swapping providers is an edit to **DEEPSEEK_URL**, **MODEL** and the key in **.env**; no Android changes.

6.6 Validation, review and undo

ActionValidator takes the parsed actions and a live snapshot of every node, returning errors prefixed **Action N:**. Its *scope* — existing nodes by lower-cased title, plus growing sets of what earlier actions will create

— is how in-batch references work. An action may declare a `ref` that later ones point at with `parentRef`, `postRef` or `fromRef`; it must *already* be registered, and registration happens *after* the parent check, so a forward reference fails rather than half-working. Titles resolve the same way, so a `set_status` on a post `create_post` proposes three rows earlier validates.

It rejects: a duplicate `ref` or an `op` outside the fourteen; more than 40 actions, a title over 200 characters, content over 50 000, a term over 80, more than 20 tags; a `create` whose title exists (live or in-batch) and an `update_post` retitling onto a taken name, which must fail here rather than inside the executor’s transaction; a parent that does not exist or is a post; a `move_post` that would cycle, checked with the `TreeRules` helper drag-and-drop uses; a status outside the enum, an unknown resource type, and missing `url`, `tag`, `term` or `definition`. A `create_link` whose `toTitle` is absent is deliberately allowed: a link to a post not yet written is legitimate, and the reader renders it unresolved.

An invalid action is not silently dropped: the validator runs over the ticked subset, re-runs on every tick, edit and relocation, and Apply stays disabled while any error stands — unticking the offending row lets the rest through. `ReviewChangesViewModel` rewrites each Action N: prefix into that action’s description and hangs it under the row. `apply()` re-validates before writing, so a stale batch cannot slip past. The screen holds the batch in memory while the user reshapes it:

- **Per-action selection.** Every row has a checkbox and an “*n* of *m* selected” counter, ticked except `delete_post`, which cascades and cannot be undone.
- **Structure.** Rows are indented by depth *inside the batch*: `ActionLabels.indent` walks `parentRef` and `parentTitle` chains with a cycle guard, anchoring at depth 0 on an existing parent, so a path renders as the tree it will become.
- **Previews and diffs.** A `create_post` row previews its Markdown through the reader’s Markwon instance; an `update_post` row a before/after from `TextDiff` — honestly primitive, a whole-document comparison, not a line diff: right for a rewrite, poor for one word.
- **Changing the destination.** Creatable and movable rows offer a picker of every branch and folder by full path (*Distributed Systems / Consensus*) plus “top level”, rewriting `parentTitle` and clearing `parentRef`.
- **Editing before saving.** Title and body can be rewritten in place; a retitle rewrites every other action addressing the old title, so tags, links and glossary entries follow the rename.
- **Confirmation for destruction.** A ticked destructive row raises a banner, and Apply a dialog naming each doomed post and its tags, links, resources and glossary entries, all beyond Undo.

Apply hands the ticked subset to `ActionExecutor`, which resolves refs to ids and runs every operation in one `db.withTransaction`. It is all-or-nothing: if operation seven throws, Room rolls back one to six and reports “*Could not apply these changes. Nothing was written.*” Content-backed `posts_fts` needs no work here (§5).

Undo is real but narrow: **in memory** (one `@Volatile UndoSnapshot` on `ChatRepository`), **per batch and single level** (a second Apply overwrites it, process death loses it), and offered from **one place**, the snackbar shown after Apply. Created nodes are deleted in reverse order, so children go before parents; nodes the batch merely touched are restored from a before-image captured inside the transaction — title, content, status, tag set, favourite flag, and parent, moved back only if it changed — taken once per node on first touch, so several edits to one post still restore its pre-batch state. Resources are removed by id; glossary terms need two cases because `addTerm` upserts, so one the batch created is deleted and one it overwrote restored.

`delete_post` is deliberately **not** undoable, and the snapshot’s header comment says so: deletion cascades to children, links, tags, resources and glossary rows, and resurrecting that sub-graph is a feature in its own right. The app says so at every step, ending with “*Undone — but deleted posts stay deleted.*”

6.7 Streaming

When health advertises **streaming**, replies arrive over SSE, and an older gateway still works on the buffered endpoint. The transport is raw `OkHttp`: Retrofit’s Gson converter buffers the whole body and leaves nothing to stream. `SseChatClient` exposes a cold `Flow<StreamFrame>` and, since a blocked socket read ignores cancellation, registers `invokeOnCompletion { call.cancel() }` so leaving the screen closes the socket; its 130s read timeout outlasts the gateway’s 120s budget, which measures the gap between chunks.

`SseFrames` is a framework-free line decoder, unit-tested on the JVM. It ignores blank lines, SSE comments (the gateway writes : `open` to flush headers early) and the `[DONE]` sentinel, and decodes `{delta}`, `{done}`,

`reply}` and `{error, retryable}`, the last two terminal. The whole reply is repeated in the `done` frame, so the envelope is parsed from one authoritative string, not re-joined deltas. A stream dying *before* the first token falls back to the buffered endpoint once; one dying later surfaces the error, since a retry would duplicate a half-rendered bubble.

Partial rendering is awkward because the wire carries JSON, not prose: the visible text is the value of `"text"` inside `{"type":"answer","text":...}`, and the buffer is not valid JSON until the final brace. Waiting would defeat streaming, so `StreamingAnswerExtractor` (plain Java) scans the buffer and classifies it as `answer`, `actions`, `context_request`, `unknown` (JSON so far, `type` not yet seen) or `prose` (no opening brace). For an answer it decodes as much of `text` as arrived, unescaping `\n`, quotes and `\uXXXX` and stopping cleanly on an escape split across chunks; for the other two it returns nothing, so raw protocol JSON is never shown and the bubble reads *“Mocha is working...”* The streaming bubble is never written to `chat_messages`, so a dead stream leaves no half-finished row.

6.8 Privacy

The assistant contradicts the local-first commitment of §1, so the boundary is drawn explicitly. **What leaves the device**, only on sending a message in the AI tab: the mode string; the session’s history (user turns and completed assistant turns — pending batches, error bubbles and other sessions are filtered out); the KB index; and the context-query results. Article text leaves only through `get_post`, for a post the model named.

What never leaves: the Room database file, any post the model did not ask for, tags, backlinks, resources, settings, backups, and the other four tabs. No analytics SDK, crash reporter or image loader is in the build (§7); **the key** lives in the gateway’s `.env` alone, the APK holds no credential, and Auto Backup is disabled.

One honest weakness: the device-to-gateway hop is cleartext HTTP, the gateway being a local process with no certificate. It is declared in `network_security_config.xml`, user-installed CAs are not trusted, and the DeepSeek hop is HTTPS; on a shared LAN the first hop is readable, and production would need TLS.

When the gateway is unreachable — for most graders, most of the time — every send begins with a health ping, and failure degrades to the banner and retry of §3. Nothing is queued or lost, and no other feature touches the network.

6.9 Where the code differs from the plan

`docs/plan.md` predates the subsystem; several decisions changed on contact with DeepSeek. §11 appends tool results as a `role: "tool"` message, calls the index “titles/ids” (`KbIndex` emits no ids) and forces an answer after three rounds where the code returns a retryable error. §12 calls streaming optional; it shipped. §13 rejects the whole batch on any validation failure, where the validator checks the ticked subset, and §14 rebuilds FTS rows in the transaction, which content-backed `posts_fts` makes unnecessary. §15 specifies a `ReviewChangesActivity`; review is a Fragment destination in the single-activity app. §22’s “Shared with AI: 3 posts” chip counts context queries, not distinct posts, and is in-memory only.

One layering shortcut: `ContextTools` reaches `nodeDao` and `knowledgeDao` directly — read-only, but the one place in the AI package that skips a repository (§11).

7 Technologies used

7.1 The stack, and why each piece is there

Technology	Version	Why this one
Kotlin	1.9.24	Primary language: coroutines and <code>Flow</code> give the whole app one asynchronous idiom, and Room’s suspend/Flow DAOs mean no database call ever needs a manual thread hop.
Java	17 target	Used deliberately for framework-free logic in <code>util/</code> : parsers, the force-directed layout, the streaming envelope reader, backup writer/reader and the diff. Pure Java classes with no Android imports are the parts worth unit-testing on the JVM, so the language boundary and the test boundary coincide.

Technology	Version	Why this one
Android SDK (XML Views)	min 24, target 35	Views rather than Compose: the design reference is a Views project, minSdk 24 keeps the reach wide, and the two custom-drawing requirements (the graph canvas, the reading column) are more direct against View and Canvas .
AGP / Gradle	8.7.3 / 8.9	Current stable pair for compileSdk 35; the wrapper is committed so the build needs no global Gradle.
Room	2.6.1	Local source of truth. Chosen over raw SQLite or files for four properties this project depends on: compile-time-checked queries, real transactions, observable Flow queries, and testable migrations. FTS4 comes from Room's @Fts4 annotation — version 2.6 has no FTS5 annotation, which is why the index is FTS4.
KSP	1.9.24-1.0.20	Room's annotation processor, roughly twice as fast as kapt on this project.
Navigation Component	2.8.3	One activity, five bottom-navigation tabs and nine pushed destinations with typed arguments and a real back stack, all declared in nav_graph.xml . Global actions keep cross-screen navigation from turning into a web of per-screen actions.
Lifecycle / ViewModel	2.8.7	ViewModel + StateFlow + repeatOnLifecycle is the app's entire state-management story; every screen exposes one immutable UI-state data class.
Material Components	1.12.0	Chips, bottom sheets, dialogs and the bottom navigation bar, all restyled onto the mocha palette rather than used with default Material colours.
Markwon	4.6.2 (core, linkify)	Renders Markdown into a TextView entirely offline. Its pluggable link resolver is what makes [[wiki-links]] clickable: they are rewritten to an internal mocha://post/<id> scheme before rendering and intercepted on tap. Only the artefacts actually used are pulled in.
Retrofit + OkHttp + Gson	2.11.0 / 4.12.0 / 2.11.0	Retrofit for the two request/response endpoints, raw OkHttp for the Server-Sent-Events stream (Retrofit has no streaming idiom worth the ceremony here), Gson for the action envelope. All three are used only for the gateway — there is no other network traffic in the app.
kotlinx-coroutines	1.9.0	Dispatchers.IO for database and network work, Dispatchers.Default for graph layout, and structured cancellation so leaving a screen cancels an in-flight AI stream.
Node.js + Express	24 / 4	The gateway. Roughly 390 lines across server.js and prompts.js , no database, no state: it holds the API key, builds the system prompt for the requested mode, normalises upstream errors, and relays the token stream.
DeepSeek API	deepseek-chat	The language model, reached only through the gateway. The app never talks to it directly and never holds a key. The action protocol is plain JSON documented in the prompt, so swapping providers means changing one URL.

Technology	Version	Why this one
JUnit 4 + Room testing	4.13.2 / 2.6.1	JVM unit tests for pure logic, plus <code>MigrationTestHelper</code> for real schema migrations with real rows.

7.2 Deliberate non-dependencies

What was left out matters as much as what went in, because every dependency is a thing that can break the build a day before a deadline:

- **No dependency-injection framework.** Hilt or Koin would earn their keep in a larger team project; here the object graph is eight lazily-created singletons on the `Application`, which is smaller and needs no annotation processor.
- **No graph library.** The knowledge graph is a `View` plus about 150 lines of Fruchterman–Reingold in Java. A charting or graph SDK would have added megabytes and a `WebView` for less control.
- **No YouTube player SDK.** Playback uses the standard `IFrame` embed in a locked-down `WebView` against `youtube-nocookie.com`, with the URL allow-list enforced in the `WebViewClient`.
- **No image loading, no analytics, no crash reporting, no ads.** Nothing in the app needs to phone home, and adding an SDK that could contradict the privacy promise made on the settings screen.
- **No Compose.** Mixing Compose into a Views codebase mid-project buys nothing the deadline can spend.

7.3 Development tooling

The repository carries an `AGENTS.md` that states the architecture rules a contributor (human or AI coding assistant) must follow: the layer boundaries, the Kotlin/Java split, the design-token rule, the phase order, and the invariant that every AI-originated *write* goes through `ActionExecutor` — read-only context lookups in `ContextTools` do query the DAOs directly, but nothing in the AI subsystem may write to one. `docs/plan.md` is the long-form design document, updated at the end of every phase with notes for the next one. Those files are development tooling, not part of the shipped application.

8 Setup and installation

There are two ways to run Learning Mocha: install the shipped APK, which needs nothing but an Android device, or build from source. The AI assistant additionally needs the small gateway process, and the app is designed so that everything else keeps working when that process is absent.

8.1 Installing the shipped APK

1. Copy `apk/app-release.apk` from the submission archive to an Android device or emulator running **Android 7.0 (API 24)** or newer.
2. Install it: `adb install -r app-release.apk`, or open the file on the device and allow installation from unknown sources.
3. Launch *Learning Mocha*. On first start the app seeds a small “Getting Started” branch so no screen is empty, and no account, permission grant or network connection is required.

The APK is signed with a self-generated release key. The keystore and its passwords are deliberately not part of the archive, so the build is reproducible without them — a source build without `keystore.properties` simply produces an unsigned release APK instead of failing.

8.2 Building the Android app from source

Prerequisites

JDK 17 (the JetBrains Runtime bundled with Android Studio is fine), the Android SDK with platform 35 and build-tools 34 or newer. No other tooling is needed — the Gradle wrapper downloads Gradle 8.9 itself.

SDK location

Create `local.properties` in the repository root with `sdk.dir=<path to your Android SDK>`. Android Studio writes this file automatically the first time the project is opened.

JDK selection

Gradle 8.9 cannot run on JDK 23 or newer. If your default `java` is newer, either point `JAVA_HOME` at a JDK 17 for the session or add `org.gradle.java.home=<path to a JDK 17>` to your own `~/.gradle/gradle.properties`. A machine-specific path is deliberately kept out of the committed `gradle.properties` so the project configures on any machine.

Listing 5: Building and testing from the repository root (PowerShell).

```
.\gradlew.bat assembleDebug      # debug APK -> app\build\outputs\apk\debug\
.\gradlew.bat installDebug      # install on a connected device or emulator
.\gradlew.bat test              # JVM unit tests
.\gradlew.bat connectedDebugAndroidTest # instrumented Room migration tests (needs a device)
.\gradlew.bat assembleRelease   # release APK -> app\build\outputs\apk\release\
```

8.3 Running the AI gateway

The gateway is a single Node process whose only job is to hold the DeepSeek API key and shape requests. It requires Node 18 or newer (developed on Node 24).

Listing 6: Starting the gateway.

```
cd backend
npm install
copy .env.example .env      # then set DEEPSEEK_API_KEY=<your key>
npm run dev                 # listens on http://localhost:8787
```

`.env` carries three settings: `DEEPSEEK_API_KEY` (required), `MODEL` (default `deepseek-chat`) and `PORT` (default 8787). The file is git-ignored and never packaged, so the key exists in exactly one place and never inside the APK.

Pointing the app at the gateway:

- **Emulator** — the default address `http://10.0.2.2:8787/` already resolves to the host machine’s `localhost`, so nothing needs configuring.
- **Physical device** — open **Settings** → **AI gateway** in the app and enter `http://<your computer’s LAN IP>:8787/`, then use **Test connection**. The device and the computer must be on the same network.

The app polls `GET /v1/health` before every request. When the gateway is unreachable the AI screens show a calm banner (“AI unavailable — your library still works”) with a retry button, and every other feature behaves exactly as it does online. Health also reports whether the gateway supports streaming, so an older gateway degrades to the non-streaming endpoint rather than failing.

8.4 Rebuilding this report

Listing 7: The report is plain $\text{\LaTeX}$ with no external image dependencies.

```
cd report
pdflatex -interaction=nonstopmode report.tex # run twice so the table of contents settles
```

8.5 Assembling the submission archive

`tools/package-submission.ps1` builds the graded archive from a clean working tree. It stages `README.md`, the full source (with `.git` history and without build output), the release APK, this PDF and the demo-video link into `dist/24125006_24125009.zip`. The script deliberately fails rather than continuing if a secret — `backend/.env`, the keystore, `keystore.properties` or anything that matches a DeepSeek key pattern — would end up inside the archive.

9 Work division

The project was built by two people. The split follows the natural seam in the architecture: one side owns the local knowledge base — storage, the tree, reading and writing posts, search, the graph, backup and the release build — and the other owns everything the assistant touches, from the gateway and the action protocol to the review screen that gates every AI write.

Student ID	Name and role	Delivered
24125006	Nguyen Anh Dung <i>Android app core</i>	Project scaffold, Gradle/AGP setup and the mocha design system (tokens, DayNight themes, typography). Room schema and all three migrations, the single-table node model, DAOs and repositories. Tree operations: create, rename, move with cycle checking, drag-to-reorder, cascade delete. Browse, Home hub, post reader and Markdown editor with live preview. Wiki-link parsing, backlinks, related posts and rename propagation. FTS4 search with filters, tags, favourites, status, dictionary storage. Knowledge-graph screen and the force-directed layout. Backup export/import, settings, theme, backup reminder. Release signing, R8 rules, adaptive launcher icon, packaging script.
24125009	Le Huu Hoa <i>AI subsystem and quality</i>	The Node/Express DeepSeek gateway: endpoint contracts, prompt construction, the four chat modes, error normalisation and Server-Sent-Events streaming. The model-independent action protocol (14 operations), its parser and the on-device validator. The seven read-only context tools and the multi-round context loop. The transactional action executor and undo snapshots. Chat UI with streamed replies and the privacy disclosure. The review-changes screen: per-action selection, tree preview, rendered previews and diffs, destination picker, edit-before-save, destructive-action confirmation. Embedded YouTube playback, dictionary CRUD screen, unit and instrumented tests, this report and the demo video.

9.1 Shared work

Some things were done together rather than divided, because splitting them would have cost more than it saved:

- **Design decisions** recorded in `docs/plan.md` — the storage model, the AI action protocol, and the list of features explicitly *not* built for version 1.
- **Code review.** Every phase ended with a review pass against the rules in `AGENTS.md` (layer boundaries, the Kotlin/Java split, design tokens, and the invariant that the AI never writes to the database directly).
- **The audit that shaped the final phase.** Before the last round of work, the whole codebase was audited against `docs/idea.md` and the grading rubric across eight dimensions (knowledge-base features, AI features, search and navigation, UI/UX, defects, release readiness, the gateway, tests and packaging). Each finding was then re-checked adversarially before anyone wrote code, which is how the two most serious problems — the tool-role request rejection and the resource-wiping save — were found and fixed rather than shipped. §11 lists what came out of it.
- **Manual testing** on the emulator: the feature matrix in §10 was walked by both members, including the offline transition and the export/import round trip.

9.2 Commit history

The repository history is included in the archive (`src/.git`), with one commit per development phase and the phase notes for the next phase written into `docs/plan.md` at the end of each one, so the reasoning

behind each step is visible in the diff rather than only in the final state.

10 Testing and verification

Testing here is deliberately lopsided. Logic that reduces to a function — parsers, the action protocol, the layout solver, the backup format — carries **125 JUnit tests** that run on the JVM in seconds with no device. The one thing that can irreversibly destroy a user’s library, a database migration, carries **2 instrumented tests** against real SQLite. The rest was verified by hand, and the last subsection says where that leaves holes.

10.1 The decision that makes the unit suite worth having

These are not smoke tests, because the interesting logic was written so it could be called without an Android runtime. `util/` is plain Java with no `android.*` import anywhere: `MarkdownLinkParser` (wiki-links and rename), `TreeRules` (move legality), `FtsQueryBuilder`, `TextDiff`, `ForceLayout` (Fruchterman–Reingold), `StreamingAnswerExtractor`, and `ExportJsonWriter` / `ImportJsonReader` / `ImportTitles`. `ai/protocol/` and `ai/engine/KbIndex` depend only on the Room *entity* data classes and Gson — never on a DAO, a `Context` or a `ViewModel` — so `ActionParser`, `ActionValidator`, `ActionLabels` and `KbIndex` are ordinary objects a test constructs with a list of `Nodes`. The language boundary and the test boundary were drawn in the same place, which is why the validator — the component standing between a language model and the database — is the most heavily tested class in the app rather than the least.

10.2 JVM unit tests

Class names below omit the `Test` suffix.

Test class	#	What it pins down
<code>ActionValidator</code>	28	Every rejection rule: forward <code>ref</code> use, duplicate titles within the batch and against the database, unknown ops, moving a node onto itself or into its own descendant, moving under a post, a post <code>ref</code> naming a container, retitling onto an existing post, unknown status and resource types, batch-size and title-length caps, and deselecting a row not renumbering the rest.
<code>BackupRoundTrip</code>	14	<code>.mocha.json</code> write-read round trip over all six knowledge tables, Unicode and Markdown bodies, node identity for foreign-key remapping, the versioned envelope, and rejection of a non-backup file, a newer version and a node with no title.
<code>ForceLayout</code>	12	Same seed gives the same layout; every node stays in the box; ring neighbours end up closer than opposite nodes; two cliques separate; isolated nodes do not stack; empty graphs, self-edges and mismatched edge arrays do not crash.
<code>ActionParser</code>	10	The three envelope kinds, fenced and bare-fenced JSON, prose stripped from around the envelope, braces inside generated Markdown surviving, and garbage falling back to a plain answer instead of throwing.
<code>ApiClient</code>	9	<code>normalizeBaseUrl</code> , kept pure precisely so it could be tested: a missing scheme assumed to be <code>http://</code> , the trailing slash Retrofit needs, path prefixes preserved, blank meaning the built-in emulator default, and unparseable input rejected rather than stored.
<code>SseFrames</code>	8	Delta ordering, the terminal frame, retryable versus non-retryable errors, escaped newlines and quotes, and malformed JSON or comment lines ignored rather than killing the stream.
<code>MarkdownLinkParser</code>	7	<code>[[wiki-link]]</code> titles and spans, case- and padding-insensitive rename, titles treated as literals not regexes, dollars and backslashes preserved, YouTube id extraction.
<code>ActionLabels</code>	6	The human label per op and review-screen indentation, including a cyclic <code>ref</code> chain terminating instead of looping.

Test class	#	What it pins down
<code>StreamingAnswerExtractor</code>	6	Growing an answer token by token out of a partial envelope, JSON escapes split across chunks, action batches staying hidden while prose passes through.
<code>KbIndex</code>	5	The titles-only outline sent with every request: nesting, sibling order, omitted status, orphaned children, and the character cap that bounds token cost.
<code>InlineResources</code>	5	Merging YouTube links found in the body with stored resource rows, without listing a video twice or giving a derived item a row id.
<code>ImportTitles</code>	5	Merge-import de-duplication: collisions detected case-insensitively, renamed <i>title (imported)</i> , then numbered.
<code>TreeRules</code>	4	Move legality: never under self, never under a descendant, siblings and root always safe.
<code>FtsQueryBuilder</code>	3	Prefix tokens joined with AND, MATCH operators stripped, punctuation-only input returning null instead of a syntax error.
<code>TextDiff</code>	3	The before/after preview on review rows, including the explicit “no text change” case.
Total	125	

10.3 Instrumented tests: Room migrations

`MigrationTest` is the only test needing a device, because it needs real SQLite. Room’s `MigrationTestHelper` builds an old database from the schemas exported to `app/schemas/`, which reach the instrumentation assets through the `androidTest.assets.srcDirs` line in `app/build.gradle`. Both tests insert real rows before migrating and assert on them after; an empty-database migration would prove nothing.

- `migrates1To3KeepingPosts` — v1 predates `exportSchema`, so there is no `1.json` to build from: the test writes the original v1 `nodes` table and indices by hand, inserts a branch and a post, then opens the result through `Room.databaseBuilder` with `MIGRATION_1_2` and `MIGRATION_2_3`. Opening through Room is the schema assertion — Room compares the live schema against v3 and throws on any mismatch. It then asserts the post’s title, status and favourite flag survived, that `posts_fts` was rebuilt *from the existing rows* rather than starting empty (a `MATCH 'spring'` query finds the pre-existing post), and that the new chat tables exist and are empty.
- `migrates2To3AddingChatTables` — builds v2 from `2.json` via the helper, inserts a post plus a tag and its join row, then runs `runMigrationsAndValidate(..., 3, true, MIGRATION_2_3)` so the helper validates the end schema against `3.json`, and asserts the tag survived and `chat_messages` exists and is empty.

The FTS assertion earns its keep: a content-backed FTS4 table added by a migration is empty until it is explicitly rebuilt, and nothing would have failed loudly — search would simply have returned nothing for every post written before the upgrade.

10.4 Running the suites

```
gradlew test                # JVM unit tests - no device, seconds
gradlew connectedDebugAndroidTest # migrations - needs an emulator or device
```

On Windows both are invoked as `.\gradlew.bat`.

10.5 Manual verification on the emulator

The *release* build — not the debug build — was installed on a Pixel emulator at API 36 and walked end to end, with the gateway from §8 running on the host and reached at `10.0.2.2:8787`. The release variant matters because it is the one whose ProGuard configuration could break Gson reflection over the AI protocol.

Scenario	Expected	Result
Install release APK	Opens on Home with the seeded library; every tab reachable with no gateway running	Pass
Live AI round trip	Real DeepSeek call through the gateway; reply streams in token by token; no action rows offered in Answer mode	Pass
AI Modify request	Batch appears on the review screen, rows inspectable and deselectable, Apply writes the approved subset in one transaction, undo offered	Pass
Unsupported op in a batch	Invalid row arrives unticked with an explanation; the valid rows still apply	Pass
Context request round	Model asks, device runs the read-only queries locally and returns results only — the library is never uploaded	Pass
Export then import	<code>.mocha.json</code> exported, app data cleared, file imported: tree, links, tags, dictionary and resources identical; chat history absent by design	Pass
Theme switching	<i>Follow system / Light / Dark</i> apply at once and survive process death, with no flash of the wrong theme	Pass
Gateway stopped	Chat shows the offline banner with a working Retry; browse, read, edit, search, graph and dictionary behave identically to the online case	Pass
Rotation on every list	Scroll position and screen state preserved	Pass

This pass is also where the most instructive defect of the project surfaced — one validator rejection disabling **Apply** for a whole batch — described with the rest of the audit findings in §11.

10.6 What is not covered

Three gaps, plainly. There are **no UI tests**: Espresso sits on the `androidTest` classpath but no fragment, adapter or navigation path has an automated test, so the review-and-apply flow — the riskiest screen in the app — is verified only by hand. There is **no automated test of the AI round trip**: it needs a live DeepSeek key and a running gateway, so it is neither hermetic nor free; the protocol is tested from the parser and validator inwards, and the network edge only through the frame decoder. And there is **no tablet-hardware pass**: the above-600 dp dimension overrides were checked on a resized emulator, never on real hardware. §11 lists these among the things a next iteration would fix first.

11 Self-assessment

This section is written to be useful rather than flattering. It records what was actually finished, what was deliberately left out, where the shipped code disagrees with the plan it was built from, and what a final review pass found wrong late enough to be worth reporting.

11.1 What was completed

Every capability described in the original concept (`docs/idea.md`) is implemented and reachable from the UI. The knowledge base — branches, folders, subfolders and Markdown posts, with create, rename, move, reorder and cascading delete — works, as do wiki-links, backlinks, related posts, tags, per-post status, favourites, external references, the two-scope dictionary, and FTS4 search across titles, content, tags, dictionary terms and resources. The AI assistant runs all four modes, reads the library only through read-only context tools, and can propose but never perform a change. Backup, import, settings and theming are done. Fourteen destinations — five tabs and nine pushed screens — are wired into a single navigation graph, and every list screen binds the same loading / empty / error state helper.

Three features were listed in the plan as explicitly *out of scope* for version 1 and were built anyway, once the graded core was safe:

- the force-directed **knowledge graph**, with a whole-library view and a one-hop neighbourhood view, laid out by the project’s own solver rather than a library;
- **streamed AI replies** over Server-Sent Events, including the awkward part — pulling a readable answer out of a JSON envelope that has only partly arrived;
- **in-app YouTube playback** in a locked-down bottom sheet, instead of throwing the user out to another app.

11.2 What was deliberately not built

None of these are accidents; each was a scope decision, and each costs something.

- **Posts are Markdown text only.** No images, no file attachments, no rich-text editing. This keeps the backup file a single self-contained JSON document with no binary side-car, and it keeps the editor honest, but it does rule out the kind of note that is mostly a diagram.
- **Videos are not embedded in the article flow.** A YouTube URL becomes a resource card below the article; tapping it opens an in-app player. The concept asked for embedding *inside* the post, which would have meant a custom Markwon plugin.
- **Drag and drop only reorders siblings.** Moving an item into a different container is a picker dialog, not a drag. Cross-container drag in a tree that is rendered one level at a time needs auto-scroll, hover-to-open and a drop-target model; the dialog does the same job.
- **The review screen edits titles and content, not metadata.** Proposed tags, status, references and dictionary entries are now *shown* on the row and can be deselected wholesale, but they cannot be edited in place before applying.
- **The assistant can delete posts, never containers.** There is no `delete_branch` or `delete_folder` operation, by design — deletion cascades, and no undo can bring a subtree back.
- **Duplicate detection in Organize mode rests on the model.** There is no on-device similarity check and no action type for merging two posts.
- **Tablet support is three dimension overrides,** not a layout. Gutters, chat-bubble width and card width change above 600 dp; there is no two-pane list-detail arrangement, and this was never verified on real tablet hardware — only on a resized emulator.
- **R8 is off in the release build.** Gson reflects over the AI protocol classes and the network DTOs, and a wrong keep rule there fails *silently*: every reply would deserialize to nulls and render as ordinary prose. The keep rules are written and correct, but the flag stays off, because a larger APK is not graded and a silently broken assistant is.

11.3 Known limits of features that do ship

- **Undo after an AI batch is narrow.** It is in-memory, single-level, and scoped to one batch: it restores title, content, status, tags, favourite and parent for the posts the batch touched, and it removes what the batch created. Deleted posts stay deleted. Leaving the review screen ends the offer.
- **The graph hides some labels rather than crowding them.** Whether labels appear at all is decided from average node spacing; on top of that, a label that would land on one already drawn is dropped, best-connected nodes winning. A dense cluster therefore reads cleanly but does not name every dot — tapping one always shows its title.
- **Search ranking is hand-rolled.** FTS4 has no `bm25`, so results are ordered in Kotlin — posts first, then title-prefix matches, then alphabetically — and snippets are built by hand rather than by SQLite.
- **The assistant needs the local gateway.** Without `backend/` running and a DeepSeek key, the chat tab shows an offline banner. Everything else in the app is unaffected, which is the point of the architecture, but it does mean the signature feature cannot be demonstrated from the APK alone.

11.4 Where the code diverged from the plan

The plan in `docs/plan.md` was written before the code and was not retro-fitted; the code is the truth. The differences worth naming:

- Context results are returned to the model as a labelled ordinary turn, not an OpenAI `tool` role — DeepSeek rejects a tool message with no matching `tool_call_id`. This also keeps the protocol portable, which was the original goal anyway.
- The context loop allows four model calls per message, not three, and a request that still needs context after the cap returns a retryable error rather than forcing an answer.
- The validator resolves titles created earlier in the same batch, not only rows already in the database — otherwise "create a post, tag it, add a term to it" could never validate, and that is the normal shape of a generated learning path.

- The backup envelope carries the six knowledge tables and not the settings, and chat history is never exported.
- `posts_fts` is content-backed, so the apply transaction does not rebuild it; Room's own triggers keep it in step with `nodes`.

11.5 What a final review pass found

Before submission the whole codebase was audited against the concept document and the grading criteria, and each finding re-checked against the code before anything was changed. The audit is the part of this project we would repeat: it found problems that testing the happy path never would.

The most instructive one was found by running the app rather than reading it. Asked to build a Distributed Systems learning path, the model returned seven operations, one of which used an operation name that does not exist in the protocol. The validator correctly rejected it — and because every row arrived ticked, one invented name disabled **Apply** for the entire batch. Six good changes were unreachable because of one bad line. Unsupported operations now arrive unticked with a plain explanation on the row, so a model that improvises costs the user that one change. Figure 4 describes the intended path; this was the case where the intended path had a sharp edge in it.

The rest, briefly:

- An AI `update_post` carrying tags *replaced* the post's whole tag set, silently dropping tags the user had added — and the review screen never displayed tags at all, so it happened unseen. Tags are now merged, and proposed tags and status are shown on the row.
- Importing the same backup twice on the merge path created duplicate post titles. Titles are how wiki-links and AI actions address posts, so duplicates make resolution arbitrary. Imported copies that collide are now given a distinct title.
- Removing a post's last tag left the tag row behind forever, listed as "0 posts" and reported to the assistant as if it existed. Empty tags are now purged.
- The tags screen read a DAO directly from its ViewModel — the one place in the app that broke the layering rule — and counted posts per tag with one query each. It now goes through the repository with a single grouped query.
- The in-app privacy text understated what is sent: a titles-only outline of the library accompanies every AI request, which the wording did not mention. It does now.
- A gateway address typed without a scheme was stored, silently ignored, and reported as unreachable forever — exactly what someone testing on a physical device would type.
- Two error-state Retry buttons did nothing, and several lists dropped their scroll position on rotation.

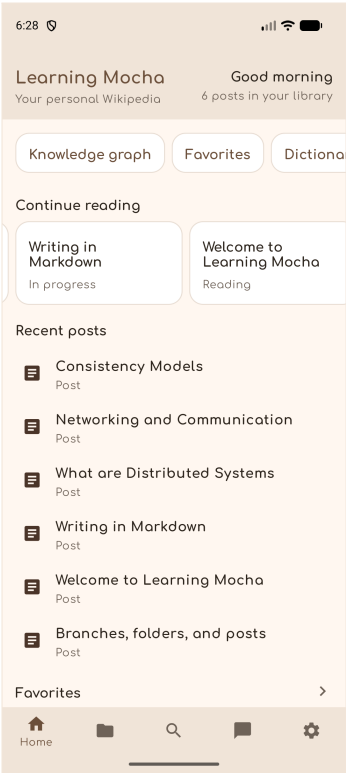
Two smaller protocol holes were closed at the same time: a reference to a branch or folder could be used where a post was required, and a reference carrying stray whitespace passed validation and then failed inside the transaction.

11.6 If there were more time

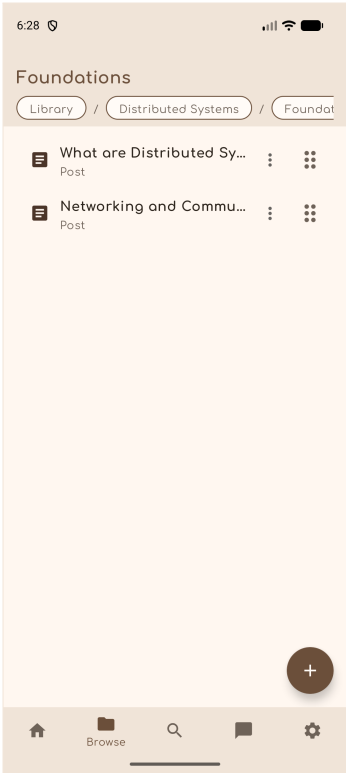
In order of what we would actually do first: a two-pane tablet layout, since it is the one rubric criterion the app answers weakly; Espresso coverage of the review-and-apply path, which is the riskiest flow in the app and currently has no automated test; editable metadata in the review screen; and cross-container drag in Browse. Further out, the interesting question is whether the action protocol could describe a merge of two posts safely enough to let Organize mode deduplicate on its own, which is the one thing the concept asked for that the model, not the app, is currently trusted to do.

A Screen gallery

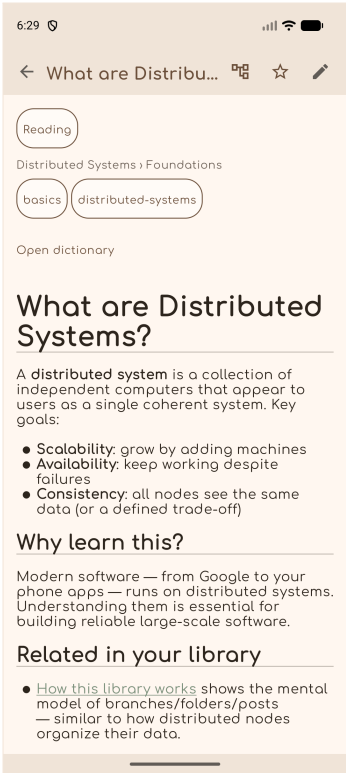
All captures come from the signed release APK running on a Pixel emulator at API 36 with the gateway live against DeepSeek. Nothing is a mock-up: the Distributed Systems branch visible in several shots was generated by the assistant from the single request quoted in §6. **Review changes** is the one to look at twice — six valid operations ticked, one the model invented left unticked with its reason, and nothing yet written to the database.



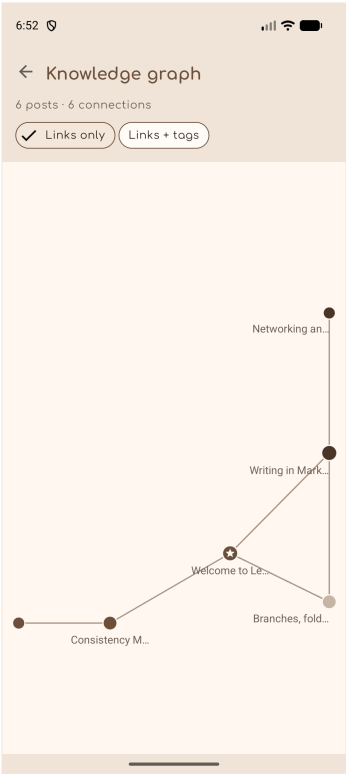
Home hub — continue reading, recent, favourites, branches.



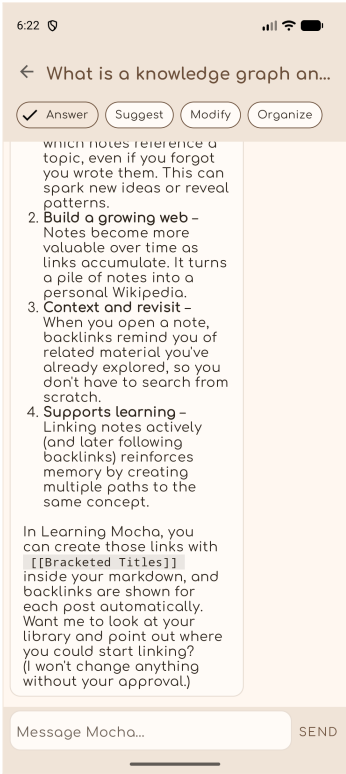
Browse — the tree with breadcrumbs; rows drag to reorder.



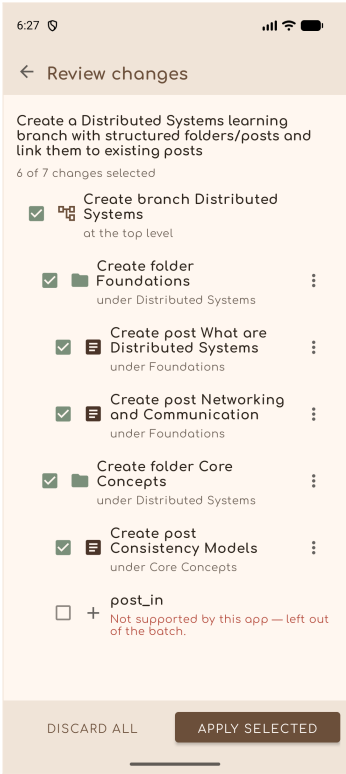
Reader — status, tags, Markdown, a live wiki-link.



Knowledge graph — whole library; the star marks a favourite.



AI chat — a streamed Answer-mode reply, rendered as Markdown.



Review changes — the gate every AI write passes through.

Not pictured, for space: the editor (title, status, tags and a Markdown toolbar over a source pane with a preview tab), search (§3), the dictionary sheet, and the offline banner the chat screen shows when the gateway is unreachable. The app is DayNight throughout — every colour above is a token with a night counterpart, and the theme follows the system setting unless overridden in Settings.